

Previsão de inadimplência e valor em risco no cartão de crédito

Projeto de Machine Learning Clássico | Tema 4.5: Default Credit Card

Otávio Bonini e Cauã Medeiros

Professor Rodrigo Ramos Silva

Inadimplência no Brasil

83,3 milhões

de brasileiros estão inadimplentes, metade da população adulta.

R\$ 568 bilhões em dívidas atrasadas

R\$ 6.814 de dívida média por pessoa

27,5% das dívidas são com bancos e cartões

Fonte: Serasa, Mapa da Inadimplência, abril de 2026.

Otávio Bonini e Cauã Medeiros | Projeto de Machine Learning

O problema

No cartão, o banco libera o dinheiro antes e só descobre o calote quando a fatura vence. E ele pode errar de dois jeitos:

Erro	Quanto custa
Deixar passar um caloteiro	O banco perde o dinheiro emprestado
Desconfiar de um cliente bom	Uma ligação de cobrança à toa

Objetivo: saber **antes** quais clientes têm mais chance de não pagar e quanto dinheiro está em jogo.

Por que não dá para fazer na mão

O Nubank tem **112 milhões** de clientes no Brasil (61% dos adultos).

Na mão

14 mil analistas

1 ano inteiro para olhar todo mundo

Quando acaba, **já está velho**

Cada analista **julga de um jeito**

Com o modelo

1 computador

Minutos para olhar todo mundo

Roda **todo mês**, sempre atualizado

A **mesma régua** para todos

Conta: $112 \text{ milhões} \times 15 \text{ min} = 28 \text{ milhões de horas} \div 2.000 \text{ horas por ano} = 14 \text{ mil analistas}$. Os 15 min são suposição nossa. Fonte: Banco Central, via Forbes, jan. 2026.

O que três pesquisas mostram

Estudo do MIT

Khandani, Kim e Lo, 2010

Testaram: modelos de árvores de decisão contra o score de crédito comum, com dados de um grande banco.

Resultado: cortando o limite de quem o modelo aponta, o banco economizaria de **6% a 25%** das perdas.

41 modelos comparados

Lessmann, Baesens, Seow e Thomas, 2015

Testaram: 41 modelos em 8 bases de crédito, entre eles **Random Forest** e **Regressão Logística**.

Resultado: o **Random Forest** foi um dos melhores, bem à frente da Logística.

O artigo da nossa base

Yeh e Lien, Taiwan, 2009

Testaram: 6 técnicas, como Regressão Logística, árvore de decisão e rede neural, **nos dados que deram origem à nossa base**.

Resultado: a **rede neural** foi a que melhor estimou a chance real de calote.

E nos bancos brasileiros?



Calcula a **chance de calote** de cada cliente antes de aumentar o limite

Blog de engenharia do Nubank (ago. 2025)



Usa mais de **1,3 mil modelos** de IA, inclusive para risco de crédito

Entrevista do diretor de tecnologia (mar. 2026)



A inadimplência de empresas caiu de **3,45% para 1,21%** depois que o banco passou a usar machine learning

Assis e Decoster, artigo científico (2025)

E desde 2025 o **Banco Central** obriga todo banco a calcular a **perda esperada**: chance de calote × quanto deve × quanto não volta.

Nossa abordagem

Classificação: qual a chance de o cliente dar calote?

Regressão: quanto ele deve?

Valor em risco = chance × valor devido. Ordenando por ele, temos a fila de cobrança.

Cliente	Chance de calote	Fatura	Valor em risco
A	80%	NT\$ 2.000	NT\$ 1.600
B	40%	NT\$ 50.000	NT\$ 20.000

O B tem metade da chance, mas vai primeiro na fila.

A base de dados

30.000 clientes de um banco de Taiwan (2005)

6 meses de histórico: atraso, fatura e pagamento

22% deram calote

Baixada da UCI com o ucimlrepo. Nenhum valor faltando.

```
import pandas as pd
[2] from ucimlrepo import fetch_ucirepo

dataset = fetch_ucirepo(id=350)
X = dataset.data.features
y = dataset.data.targets

print(dataset.metadata['num_instances'], 'linhas')
print(dataset.metadata['num_features'], 'atributos')
print('valores ausentes:', dataset.metadata['has_missing_values'])

30000 linhas
23 atributos
valores ausentes: no
```

Tratamento dos dados: limpeza e colunas novas

Limpeza: tiramos códigos que não existem na documentação

399 linhas saíram (1,33%)

4 colunas novas resumindo os 6 meses

meses com atraso, pior atraso, uso do limite e pagamento médio

```
antes = len(df)
[9] df = df[~df['EDUCATION'].isin([0, 5, 6])]
df = df[df['MARRIAGE'] != 0]
df = df.copy()
print(f'Removidas {antes - len(df)} linhas ({(antes-len(df))/antes*100:.2f}%)')
print(f'Taxa de inadimplência após remoção: {df["inadimplente"].mean()*100:.2f}%)'
```

Removidas 399 linhas (1.33%)

Taxa de inadimplência após remoção: 22.31%

```
[11] PAY = [f'PAY_{i}' for i in range(1, 7)]
BILL = [f'BILL_AMT_{i}' for i in range(1, 7)]
PAG = [f'PAY_AMT_{i}' for i in range(1, 7)]

# Em quantos dos 6 meses o cliente atrasou
df['n_atrasos'] = (df[PAY] >= 1).sum(axis=1)

# Pior atraso do período
df['max_atraso'] = df[PAY].max(axis=1)

# Quanto da fatura ele usa do limite disponível
df['util_media'] = df[BILL].mean(axis=1) / df['LIMIT_BAL']

# Quanto ele paga, em média
df['pgto_medio'] = df[PAG].mean(axis=1)
```

Tratamento dos dados: treino, teste e escala

80% treino e **20%** prova

stratify: **22,31%** de calote nos dois

StandardScaler: números na mesma escala

One-Hot: categorias viram colunas de 0 e 1

```
[13] from sklearn.model_selection import train_test_split

X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

print(f'Treino: {X_tr.shape[0]} ({y_tr.mean()*100:.2f}% inadimplentes)')
print(f'Teste: {X_te.shape[0]} ({y_te.mean()*100:.2f}% inadimplentes)')
```

Treino: 23680 (22.31% inadimplentes)
Teste: 5921 (22.31% inadimplentes)

```
[14] from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

CATEGORICAS = ['SEX', 'EDUCATION', 'MARRIAGE']
NUMERICAS = [c for c in X.columns if c not in CATEGORICAS]

prep = ColumnTransformer([
    ('num', StandardScaler(), NUMERICAS),
    ('cat', OneHotEncoder(drop='first', handle_unknown='ignore'), CATEGORICAS),
])

print(f'{len(NUMERICAS)} numéricas para o StandardScaler')
print(f'{len(CATEGORICAS)} categóricas para o One-Hot')
```

24 numéricas para o StandardScaler
3 categóricas para o One-Hot

O que os dados mostraram

```
df.groupby('PAY_1')['inadimplente'].agg(['mean', 'count'])
```

	mean	count
PAY_1		
-2	0.132	2759
-1	0.168	5686
0	0.128	14737
1	0.339	3688
2	0.691	2667
3	0.758	322
4	0.684	76
5	0.500	26
6	0.545	11
7	0.778	9
8	0.579	19

Taxa de calote por atraso no último mês:

Em dia: **16,8%**

1 mês de atraso: **33,9%**

2 meses de atraso: **69,1%**

2 meses de atraso = **4 vezes** mais calote

-2 e 0: cartão sem uso e pagamento mínimo

Modelos testados

Vai dar calote?

Logística e **Random Forest**

Quanto deve?

Linear, Polinomial e **Random Forest**

GridSearchCV: testa as configurações e fica com a melhor

	MAE	RMSE	R ²
Modelo			
Random Forest Regressor	7565.770	20021.022	0.920
Polinomial (grau 2)	8786.872	20868.967	0.913
Linear Múltipla	8547.735	21270.094	0.910

```
[17] from sklearn.model_selection import GridSearchCV, StratifiedKFold

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

grid_lr = GridSearchCV(
    Pipeline([('prep', prep), ('clf', LogisticRegression(max_iter=2000,
        class_weight='balanced', random_state=42))]),
    {'clf__C': [0.01, 0.1, 1.0]},
    scoring='f1', cv=cv, n_jobs=-1
)
grid_lr.fit(X_tr, y_tr)
print('LogReg:', grid_lr.best_params_, f'F1(CV) = {grid_lr.best_score_:.4f}')

grid_rf = GridSearchCV(
    Pipeline([('prep', prep), ('clf', RandomForestClassifier(n_estimators=200,
        class_weight='balanced_subsample', n_jobs=-1, random_state=42))]),
    {'clf__max_depth': [10, 16, None], 'clf__min_samples_leaf': [5, 20]},
    scoring='f1', cv=cv, n_jobs=-1
)
grid_rf.fit(X_tr, y_tr)
print('RF:', grid_rf.best_params_, f'F1(CV) = {grid_rf.best_score_:.4f}')

LogReg: {'clf__C': 1.0} F1(CV) = 0.5246
RF: {'clf__max_depth': None, 'clf__min_samples_leaf': 20} F1(CV) = 0.5521
```

Resultados na prova (5.921 clientes)

Modelo	Acurácia	Precision	Recall	F1	ROC-AUC
Regressão Logística	74.33	44.64	62.68	52.14	76.10
Random Forest	76.90	48.55	59.42	53.44	77.98
[[3768 832]					
[536 785]]					
	precision	recall	f1-score	support	
Adimplente	0.88	0.82	0.85	4600	
Inadimplente	0.49	0.59	0.53	1321	
accuracy			0.77	5921	
macro avg	0.68	0.71	0.69	5921	
weighted avg	0.79	0.77	0.78	5921	

Acurácia: **76,9%**

Chutar "ninguém dá calote" daria 78%. Sozinha, ela engana.

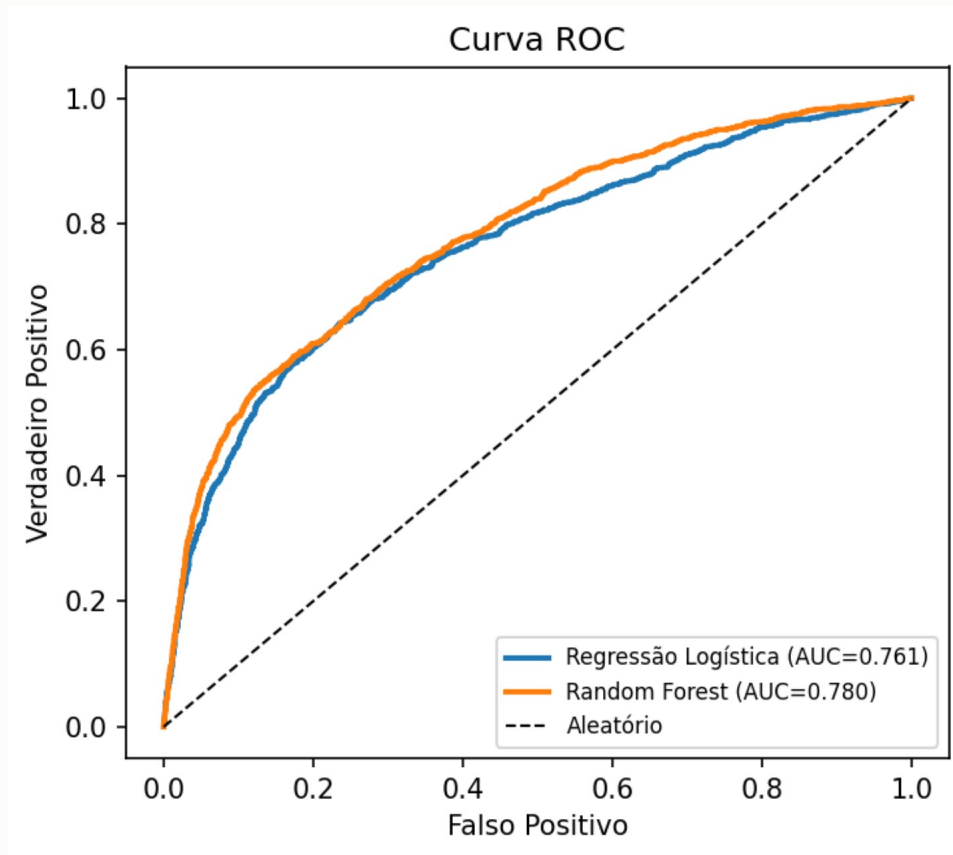
Recall: **59,4%**

785 de 1.321 caloteiros

ROC-AUC: **0,78**

536 caloteiros escaparam

Curva ROC e escolha do modelo



Random Forest: **0,780**

Regressão Logística: **0,761**

Quanto mais longe da linha tracejada (chute), melhor.

Random Forest: acerta um pouco mais, mas é difícil explicar por que recusou um cliente.

Regressão Logística: acerta um pouco menos, mas mostra o peso de cada informação.

Escolhemos o Random Forest. Num banco real, essa troca teria que ser pesada.

Valor em risco e fila de cobrança

Clientes na carteira de teste : 5921
Exposição total real : NT\$ 290,969,087
Valor em Risco total estimado : NT\$ 103,362,408 (35.5% da exposição)

OS 10% MAIS CRÍTICOS:

taxa real de inadimplência: 40.7% (carteira inteira: 22.3%)
lift: 1.82x

Top 10 clientes para ação preventiva:

	p_inadimplencia	exposicao_estimada	exposicao_real	valor_em_risco	inadimplente_real
11855	0.85	548142.91	520453.0	466821.49	0
27467	0.82	562761.56	491810.0	462269.39	1
5924	0.80	566432.99	581775.0	452482.61	1

```
# Probabilidade de calote
[22] p_default = rf_final.predict_proba(X_te)[: , 1]

# Exposição estimada, nos mesmos clientes
reg_final = modelos_reg['Random Forest Regressor']
exposicao = reg_final.predict(Xr_te)

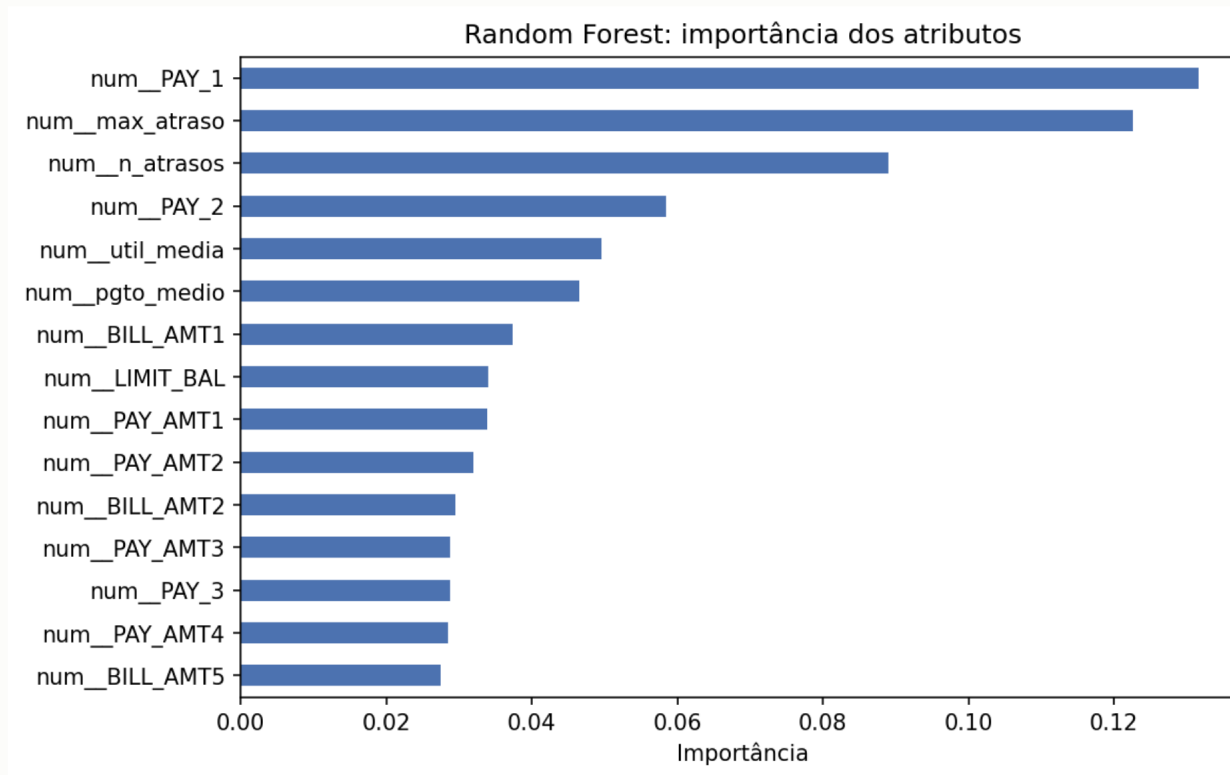
# Valor em Risco = probabilidade x exposição
valor_risco = p_default * np.clip(exposicao, 0, None)
```

Top 10% da fila: **40,7%** de calote
(média: 22,3%)

1,82 vez mais caloteiros do que
ao acaso

Em amarelo: o 2º da fila, com 82% de
chance. Deu calote (1).

O que mais pesou no modelo



As 6 primeiras são sobre **pagamento**

4 delas foram criadas por nós

Idade, sexo e estado civil ficam no fim

"num_" é o prefixo do ColumnTransformer

Limitações

Dados antigos e de fora: Taiwan, 2005. Para o Brasil de hoje, teria que treinar de novo.

Valor aproximado: não sabemos quanto o banco recupera depois do calote.

Só quem já tem cartão: o modelo não sabe nada de quem foi recusado.

Dados sensíveis: sexo e estado civil estão no modelo. Usar isso para decidir crédito é questionável.

Conclusão: como um banco usaria

1. Todo mês, o modelo olha todos os clientes em segundos e monta uma fila pelo **valor em risco**.
2. A cobrança começa pelo topo: nos 10% primeiros, **41%** dão calote, quase o dobro da média.
3. O modelo só organiza a fila. Quem decide o que fazer continua sendo uma pessoa.

Obrigado!

Otávio Bonini e Cauã Medeiros | Código e relatório: github.com/otaviobonini/machinelearningprojeto

Fontes

[Serasa: Mapa da Inadimplência \(abr. 2026\)](#)

serasa.com.br

[Khandani, Kim e Lo \(2010\), Journal of Banking & Finance](#)

web.mit.edu

[Yeh e Lien \(2009\), Expert Systems with Applications](#)

researchgate.net

[Building Nubank: modelos de risco para aumento de limite \(ago. 2025\)](#)

building.nu.com

[Assis e Decoster \(2025\): o caso do Banco BS2](#)

redalyc.org

[Forbes: Nubank passa de 112 milhões de clientes \(jan. 2026\)](#)

forbes.com.br

[Lessmann et al. \(2015\), European Journal of Operational Research](#)

sciencedirect.com

[Base de dados: UCI, Default of Credit Card Clients](#)

archive.ics.uci.edu

[Let's Money: Itaú usa mais de 1,3 mil modelos de IA \(mar. 2026\)](#)

letsmoney.com.br

[Deloitte: Resolução CMN 4.966](#)

deloitte.com